

Laporan Praktikum 4
Aplikasi Mobile



Oleh: Iqbal Miftahul Fikri
NIM 2411531004

Dosen Pengampu : Nurfiah, S.ST, M.Kom

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
Padang
2026

A. Pendahuluan

Pengembangan aplikasi mobile modern bertolak dari kemampuan antarmuka dalam menyajikan informasi secara terstruktur melalui beberapa layar terpisah. Dalam framework Flutter, pengelolaan alur perpindahan antarlayar dan pertukaran data antarkomponen menjadi fondasi utama untuk membangun pengalaman pengguna yang interaktif. Pada aplikasi Expense Tracker, alur navigasi dari halaman utama menuju halaman formulir pencatatan maupun halaman rincian transaksi memerlukan mekanisme pengisian data yang terstruktur dan aman.

Praktikum ini berfokus pada penerapan konsep tumpukan (stack) melalui widget Navigator bawaan Flutter, di mana instruksi Navigator.push digunakan untuk menumpuk rute layar baru dan Navigator.pop untuk menghapus layar teratas guna kembali ke halaman sebelumnya. Selain itu, praktikum ini melatih pengimplementasian passing data antar-widget memanfaatkan variabel final dan constructor berklasa required, serta penambahan komponen interaktif berupa date picker berbasis showDatePicker dan validasi formulir input transaksi.

B. Tujuan

Tujuan dari praktikum ini adalah:

- Memahami dan mengimplementasikan konsep tumpukan (stack) pada sistem navigasi antarmuka Flutter menggunakan kelas Navigator.
- Menerapkan perintah Navigator.push untuk berpindah ke layar baru dan Navigator.pop untuk menutup halaman formulir setelah aksi penyimpanan selesai.
- Mengimplementasikan pengiriman data antar-layar memanfaatkan variabel final dan constructor dengan penanda required.
- Mengintegrasikan komponen calendar picker menggunakan fungsi asynchronous showDatePicker pada kolom penginputan tanggal transaksi.
- Menerapkan fungsi validasi data input pada formulir transaksi menggunakan GlobalKey<FormState> dan menampilkan notifikasi umpan balik menggunakan SnackBar.

C. Langkah-Langkah

1. Membuat Halaman Detail Transaksi.

Buat file baru dengan nama detail_transaction_screen.dart. Lalu buat kode seperti ini.

```

class DetailTransactionScreen extends StatelessWidget {
  final String title;
  final String amount;

  // Parameter wajib diserahkan dari layar sebelumnya
  const DetailTransactionScreen({
    super.key, required this.title, required this.amount
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Detail
Transaction")),
      body: Center(
        child: Column(
          children: [
            Text(title, style: TextStyle(fontSize: 24)),
            Text(amount, style: TextStyle(fontSize: 20)),
          ],
        ),
      ),
    );
  }
}

```

Kode ini mendefinisikan sebuah widget Stateless bernama DetailTransactionScreen yang berfungsi untuk menampilkan halaman detail transaksi dengan wajib menerima kiriman data judul dan nominal dari layar sebelumnya melalui constructor, lalu menampilkan kedua informasi teks tersebut secara vertikal tepat di tengah layar menggunakan kombinasi widget Scaffold, Center, dan Column.

2. Pindah ke Detail & Mengirim Data (Push).

Buka kembali Dashboard. Pada widget ListTile daftar transaksi, tambahkan fungsi onTap.

```

ListTile(
  leading: Icon(Icons.fastfood),
  title: Text('Makan Siang'),
  trailing: Text('- Rp 50.000'),
  onTap: () {
    // Action saat baris di-klik
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) =>
          DetailTransactionScreen(
            title: 'Makan Siang', // Mengirim Data
            amount: '- Rp 50.000', // Mengirim Data
          ),
      ),
    );
  },
);

```

Kode ini membuat sebuah komponen baris daftar transaksi (ListTile) dengan ikon makanan, judul "Makan Siang", dan nominal harga, di mana ketika pengguna menekan baris tersebut, fungsi Navigator.push akan dijalankan untuk membuka halaman DetailTransactionScreen sekaligus menyertakan data judul dan nominal transaksi ke halaman tujuan.

3. Menutup Layar / Kembali Otomatis (Pop).

Kembali ke file add_transaction_screen.dart. Kita ingin formulir otomatis tertutup dan kembali ke Dashboard setelah klik Simpan.

```

ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      // Tampilkan Notifikasi Sukses
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Menyimpan Transaksi...'))
      );

      // EKSEKUSI POP
      // Menutup form ini dan kembali ke layar sebelumnya
      Navigator.pop(context);
    }
  },
  child: const Text("Simpan Transaksi"),
);

```

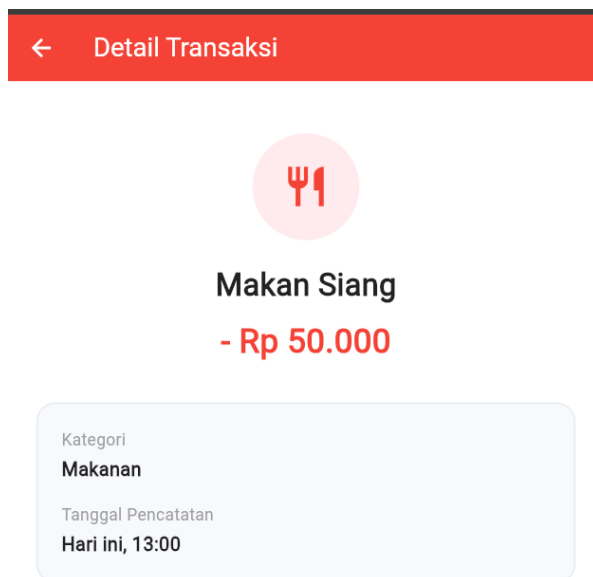
Kode ini membuat tombol aksi bernama "Simpan Transaksi" yang ketika ditekan akan memvalidasi seluruh isian formulir terlebih dahulu, jika data sudah valid,

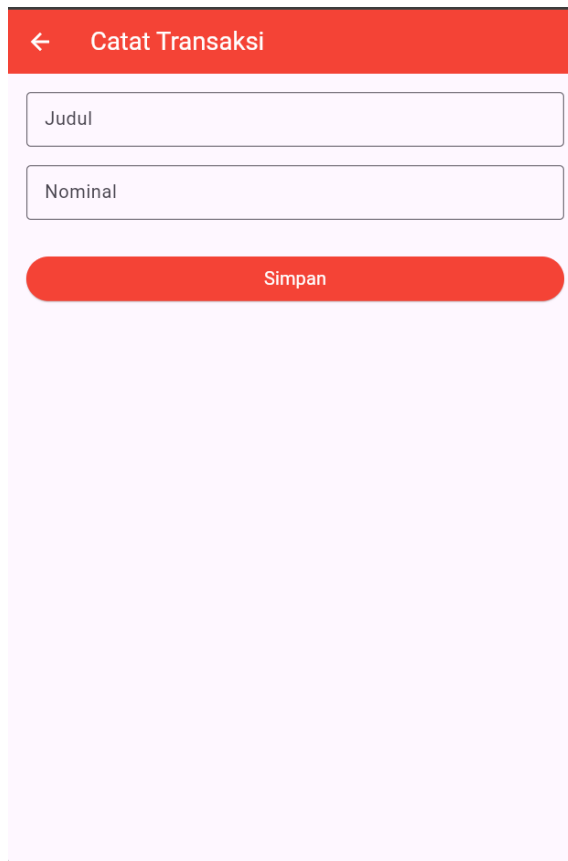
aplikasi akan menampilkan notifikasi pemberitahuan melayang bertuliskan "Menyimpan Transaksi..." kemudian menutup layar formulir tersebut untuk kembali ke halaman utama menggunakan perintah `Navigator.pop(context)`.

4. Hasil Akhir : Mockup Halaman Detail.

Sekarang daftar transaksi di Dashboard bisa di-klik dan akan membuka detail yang berisi data lengkap.

Jika sudah selesai maka akan tampil output seperti ini:





Latihan/Tugas

Input Tanggal

Tambahkan satu input baru sebelum tombol Simpan untuk mengisi Tanggal Transaksi.

- Bisa menggunakan TextFormField biasa (format teks DD/MM/YYYY).
- Tombol plus: gunakan fungsi showDatePicker.
- Berikan validasi wajib diisi!

5. Menambahkan Controller Tanggal.

Tambahkan sebuah TextEditingController baru khusus untuk menampung teks tanggal.



```
// 2411531004

final _dateController =
  TextEditingController();
```

Controller ini berfungsi sebagai penampung dan pengelola data teks tanggal yang akan ditampilkan pada kolom input setelah pengguna memilih tanggal di kalender.

6. Membuat Fungsi Pemilih Tanggal (showDatePicker).

Buat fungsi untuk memunculkan modal kalender bawaan Flutter.



Fungsi `showDatePicker` akan menampilkan popup kalender. Jika pengguna memilih tanggal, tanggal tersebut akan diubah formatnya menjadi DD/MM/YYYY dan nilainya dimasukkan ke dalam `_dateController`.

7. Menambahkan TextFormField Tanggal dan Validasi.

Tambahkan komponen kolom input tanggal tepat di atas tombol Simpan.



Penambahan komponen TextFormField ini dikonfigurasi dengan properti `readOnly: true` agar pengguna tidak dapat mengetikkan tanggal secara manual, melainkan diwajibkan memilihnya langsung dari tampilan modal kalender. Event `onTap` pada kolom input serta tombol `suffixIcon` dihubungkan ke fungsi `_selectDate` sehingga popup kalender akan otomatis muncul saat kolom atau ikon kalender diklik. Selain itu, bagian validator bertugas melakukan pengecekan data dan akan menampilkan pesan peringatan "Tanggal wajib diisi!" apabila tombol simpan ditekan ketika kolom tanggal masih dalam kondisi kosong.

8. Hasil Akhir.

Jika sudah selesai, maka akan tampil output seperti ini:

← Catat Transaksi

Judul

Tidak boleh kosong

Nominal

Tidak boleh kosong

Tanggal Transaksi



Tanggal wajib diisi!

Simpan

Select date

Wed, Sep 30



September 2026 ▼



S	M	T	W	T	F	S
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Cancel

OK

D. Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa pengelolaan navigasi antarmuka pada Flutter bekerja berbasis struktur data tumpukan yang dikelola secara efisien oleh widget Navigator. Metode Navigator.push terbukti efektif untuk membuka halaman baru seperti formulir transaksi atau detail transaksi, sedangkan Navigator.pop memungkinkan aplikasi menutup layar teratas dan mengembalikan kontrol ke halaman sebelumnya tanpa merusak state aplikasi. Pengiriman data antar-layar menggunakan variabel final yang dipassing melalui constructor tipe named parameter berklausula required mampu menjamin ketersediaan data secara aman dan konsisten dari halaman Dashboard ke halaman Detail Transaksi.

Selain sistem navigasi, pengimplementasian formulir interaktif pada aplikasi Expense Tracker berhasil ditingkatkan fungsinya dengan penambahan fitur pemilih tanggal menggunakan showDatePicker dan penguncian input manual melalui readOnly: true. Mekanisme validasi formulir melalui _formKey.currentState!.validate() memastikan bahwa seluruh data transaksi yang wajib diisi, termasuk judul, nominal, dan tanggal, telah tervalidasi sebelum proses penyimpanan dieksekusi. Kombinasi antara struktur navigasi yang tepat, pertukaran data yang aman, serta validasi formulir menghasilkan arsitektur aplikasi mobile yang solid, rapi, dan responsif.